Continental University of Florida

Assignment N.° 2

Modelo de simulación discreta de una línea de ensamblaje de placas electrónicas en AnyLogic

Tonatiuh Hernan Ovidio Cadenillas Cáceres

Resumen

Redistribuir los recursos de la línea (pasar de tres operarios y una mesa de pruebas a dos operarios y dos mesas) aumentó la producción del turno, pero no resolvió el problema de capacidad: solo trasladó el cuello de botella de la estación de pruebas a la de ensamblaje. La configuración que conviene evaluar a continuación es la de tres operarios con dos mesas.

Para llegar a esta conclusión se construyó en AnyLogic un modelo de simulación de eventos discretos de una línea de ensamblaje de placas electrónicas. El modelo incluye una entidad (el agente Placa) con dos atributos, dos colas que representan el inventario en proceso, dos recursos (operarios y mesas de prueba) y dos eventos programados que simulan la falla y reparación de una mesa. Se ejecutaron dos experimentos de 480 minutos (un turno). En el escenario optimizado las placas terminadas subieron de 382 a 472 (23,6 % más), el tiempo medio en el sistema bajó de 120,79 a 71,53 minutos y la cola media ante pruebas cayó de 163,18 a 0,08 placas. Al cierre del turno, sin embargo, había 246 placas esperando ensamblaje, y los operarios trabajaban al 99 % de su capacidad mientras las mesas lo hacían al 54 %. El informe documenta el marco teórico, los materiales y métodos, la lógica del modelo, los experimentos, los resultados y las limitaciones detectadas en la verificación.

Palabras clave: simulación de eventos discretos, AnyLogic, línea de ensamblaje, teoría de colas, cuello de botella.

Introducción

En el ensamblaje de placas electrónicas, cada placa pasa por estaciones donde

compite por recursos limitados (operarios, equipos de prueba) y espera en colas cuando esos recursos están ocupados. La variabilidad de las llegadas y de los tiempos de proceso, junto con fallas eventuales de los equipos, hace difícil predecir con cálculos estáticos cuántas placas se terminarán en un turno o dónde se acumulará el inventario en proceso. La simulación de eventos discretos permite reproducir ese comportamiento en el tiempo y comparar alternativas de asignación de recursos sin alterar la operación real.

Este informe complementa el avance entregado anteriormente. Añade el marco teórico que sustenta el proyecto, la descripción de los materiales y métodos usados para implementar el modelo, y el análisis de los experimentos realizados.

Marco teórico

Conceptos básicos de la simulación

Simular consiste en construir un modelo de un sistema real y experimentar con él para entender su comportamiento o evaluar estrategias de operación (Barceló, 1996). Un sistema es un conjunto de elementos que interactúan para cumplir un objetivo; el modelo es una representación simplificada que conserva solo los elementos necesarios para responder las preguntas del estudio. La simulación se usa cuando el sistema es demasiado complejo para resolverse analíticamente, cuando experimentar con el sistema real es costoso o riesgoso, o cuando interesa observar su evolución en el tiempo y no solo un resultado promedio (Law, 2015).

Historia y evolución de la simulación

El método Monte Carlo, desarrollado en la década de 1940 por Ulam y von Neumann, introdujo el uso de números aleatorios para estudiar sistemas con incertidumbre. En los años sesenta aparecieron los primeros lenguajes de simulación de propósito específico, como GPSS (desarrollado por Geoffrey Gordon en IBM) y SIMSCRIPT, que permitieron describir sistemas de colas sin programar desde cero la gestión del tiempo.

A partir de los noventa se difundieron las herramientas gráficas con animación, y hoy entornos como AnyLogic combinan en un mismo modelo la simulación de eventos discretos, la dinámica de sistemas y el modelado basado en agentes.

Importancia y aplicaciones en Ingeniería Industrial

En Ingeniería Industrial la simulación se aplica al diseño y balanceo de líneas de producción, la planificación de capacidad, la gestión de inventarios, la logística, los centros de servicio y los sistemas de salud. Su principal ventaja es que permite probar escenarios ("¿qué pasa si se agrega una mesa de pruebas?") antes de invertir recursos, e identificar cuellos de botella que no son evidentes en un análisis de promedios.

Tipos de simulación: continua y discreta

En la simulación continua, las variables de estado cambian de forma continua en el tiempo y se describen con ecuaciones diferenciales; es el enfoque de la dinámica de sistemas (niveles y flujos). En la simulación discreta, las variables de estado cambian solo en instantes específicos, llamados eventos, y entre dos eventos el estado permanece constante. El reloj de simulación avanza de un evento al siguiente, sin recorrer los intervalos en los que no ocurre nada (Banks et al., 2010).

La línea de ensamblaje se modeló con simulación discreta porque su estado (número de placas en cada cola, operarios ocupados, estado de la mesa) cambia únicamente cuando llega una placa, cuando empieza o termina una operación, o cuando la mesa falla o se repara.

Conceptualización de sistemas y modelado de sistemas dinámicos

Un sistema dinámico es aquel cuyo estado evoluciona en el tiempo. Su estado en cada instante se describe con variables de estado, y su evolución depende del estado actual y de las entradas. En la línea modelada, las variables de estado son el tamaño de cada cola, la cantidad de recursos ocupados, la capacidad disponible de la mesa de pruebas

y los contadores acumulados.

Aunque el modelo es discreto, cada cola se comporta como un nivel (stock) en el sentido de la dinámica de sistemas: aumenta con el flujo de entrada (llegadas) y disminuye con el flujo de salida (placas atendidas). Cuando la tasa de entrada supera de forma sostenida la capacidad de salida, la cola crece aproximadamente a una velocidad igual a la diferencia entre ambas tasas. Esta relación explica el crecimiento casi lineal de las colas que se observa en los gráficos de resultados.

Entidades, atributos, recursos, colas y eventos

Los modelos de eventos discretos se describen con cinco elementos básicos (Banks et al., 2010; Barceló, 1996):

- **Entidad:** objeto que fluye por el sistema y cuyo recorrido interesa estudiar, como una placa o un cliente.
- **Atributo:** característica propia de cada entidad que puede influir en su tratamiento o servir para recolectar estadísticas, como el tipo de placa o la hora de llegada.
- **Recurso:** elemento de capacidad limitada que las entidades necesitan para avanzar, como un operario o una mesa de pruebas. Puede estar libre, ocupado o fuera de servicio.
- **Cola:** lugar donde las entidades esperan cuando el recurso que necesitan está ocupado. En manufactura, las colas representan el inventario en proceso (WIP).
- **Evento:** suceso instantáneo que cambia el estado del sistema, como una llegada, el fin de un servicio o la falla de un equipo.

Indicadores de teoría de colas

La utilización de una estación con *c* servidores se calcula como $\rho = \lambda / (c \cdot \mu)$, donde λ es la tasa de llegada y μ la tasa de servicio de cada servidor. Si ρ es mayor o igual a 1, la estación no tiene capacidad suficiente y la cola crece sin límite. La ley de Little, $L = \lambda \cdot W$, relaciona el número medio de entidades en el sistema (L) con el tiempo medio de

permanencia (W) cuando el sistema está en estado estable. Ambas relaciones se usaron para verificar los resultados del modelo.

Distribuciones de probabilidad empleadas

- **Exponencial:** describe tiempos entre llegadas aleatorias e independientes. En AnyLogic, exponential(λ) recibe la tasa λ y su media es 1/λ.
- **Triangular:** se define con mínimo, moda y máximo. Es útil cuando no hay datos históricos y se trabaja con estimaciones de expertos. En AnyLogic se escribe triangular(min, moda, max).
- **Normal:** describe tiempos simétricos alrededor de una media. En AnyLogic la función es normal(σ, μ): el primer argumento es la desviación estándar y el segundo la media. Como puede generar valores negativos, en tiempos de proceso suele truncarse.

Construcción de modelos: AnyLogic

AnyLogic es un software de simulación multimétodo desarrollado por The AnyLogic Company (s. f.). Su Process Modeling Library ofrece bloques predefinidos para modelar procesos: Source (genera entidades), Queue (cola), Seize y Release (toman y liberan recursos), Delay (retardo), Service (combina Seize, Delay y Release), ResourcePool (conjunto de recursos), TimeMeasureStart y TimeMeasureEnd (miden tiempos) y Sink (elimina entidades). Además permite definir agentes con variables propias, parámetros para configurar escenarios, eventos programados con código Java y experimentos de simulación con semilla aleatoria fija o variable.

Descripción del sistema

El sistema representa una línea de ensamblaje de placas electrónicas durante un turno de 8 horas (480 minutos). Las placas llegan de forma aleatoria y esperan en una cola hasta que un operario de ensamblaje queda libre. El operario ensambla la placa y, al terminar, queda disponible para la siguiente. La placa pasa luego a una segunda cola,

donde espera una mesa de pruebas de calidad. Tras la prueba, la placa sale del sistema como producto terminado.

A mitad del turno (minuto 240) una mesa de pruebas falla y permanece fuera de servicio 15 minutos, hasta el minuto 255. El 25 % de las placas se clasifica como de alta complejidad; en esta versión del modelo el tipo se registra para las estadísticas, pero no cambia los tiempos de proceso.

Supuestos del modelo:

- El sistema empieza vacío en el minuto 0 y no se aplica periodo de calentamiento.
- Las colas tienen capacidad ilimitada y atienden en orden de llegada (FIFO).
- Cada placa usa un operario durante el ensamblaje y una mesa durante la prueba.
- No hay tiempos de traslado entre estaciones ni reprocesos por fallas de calidad.
- No se modelan costos.

Materiales y métodos

Materiales

- AnyLogic Personal Learning Edition, con la Process Modeling Library.
- Computadora personal con sistema operativo Windows.
- Git y un repositorio remoto para versionar y publicar el proyecto.
- Material del curso: videos "How-To" de AnyLogic (AnyLogic, 2024), Barceló (1996) y la documentación oficial de AnyLogic.

Metodología

El proyecto siguió las etapas clásicas de un estudio de simulación (Banks et al., 2010):

1. **Formulación del problema:** determinar qué asignación de operarios y mesas mejora la producción del turno.
2. **Conceptualización:** identificación de entidades, atributos, recursos, colas y eventos, y definición de supuestos.

3. **Datos de entrada:** definición de las distribuciones de llegadas y de tiempos de proceso, y de los tiempos de falla y reparación.
4. **Traducción del modelo:** construcción del diagrama de flujo en AnyLogic y programación de acciones y eventos.
5. **Verificación:** revisión del balance de entidades, de la coherencia con cálculos de capacidad y de la sintaxis de las distribuciones.
6. **Diseño de experimentos:** definición de dos escenarios con semilla fija.
7. **Ejecución y análisis:** corridas de 480 minutos, registro de indicadores y comparación.
8. **Documentación:** redacción del informe y publicación del proyecto en el repositorio GIT.

Identificación de los elementos del modelo

El enunciado exige al menos dos de los siguientes elementos: colas, inventarios, recursos y eventos. El modelo incluye los cuatro. La Tabla 1 muestra cómo se implementó cada concepto teórico y por qué se eligió.

Tabla 1

Elementos del modelo y su relación con los conceptos teóricos

Concepto	Implementación en AnyLogic	Justificación
Entidad	Agente Placa, generado por el bloque llegadaPlacas	Es el objeto que recorre la línea; su flujo determina la producción y el WIP.
Atributos	tipoPlaca (String: "Alta" o "Baja") y horaLlegada (double)	tipoPlaca permite clasificar la mezcla de producción; horaLlegada permite calcular el tiempo de cada placa en el sistema.
Recursos	ResourcePool	Son las capacidades limitadas

Concepto	Implementación en AnyLogic	Justificación
	Operarios_Ensamblaje y Mesa_Pruebas	cuya asignación se quiere evaluar; su cantidad es la variable de decisión de los escenarios.
Colas / inventario	Queue Cola_Ensamblaje y Cola_Pruebas	Representan el inventario en proceso antes de cada estación y permiten ubicar el cuello de botella.
Eventos de proceso	Llegada (Source), inicio y fin de ensamblaje (Seize/Delay/Release), inicio y fin de prueba (Service), salida (Sink)	Son los instantes en que cambia el estado del sistema y avanza el reloj de simulación.
Eventos programados	falloMesaCada8h y finReparacionMesa	Introducen una perturbación que reduce temporalmente la capacidad, como ocurre con fallas reales de equipos.
Variables de estado	Tamaño de colas, recursos ocupados, capacidad de Mesa_Pruebas, mesaEnReparacion	Describen el estado dinámico del sistema en cada instante.

Datos de entrada y parámetros

Tabla 2

Datos de entrada configurados en el modelo

Elemento	Bloque	Expresión en AnyLogic	Unidad	Interpretación en AnyLogic
Tiempo entre	llegadaPlacas	exponential(1.5)	minutos	Tasa de 1,5

Elemento	Bloque	Expresión en AnyLogic	Unidad	Interpretación en AnyLogic
llegadas	(Source)			placas/min; media de 0,67 min entre llegadas (unas 720 placas por turno)
Tiempo de ensamblaje	Ensamblaje (Delay)	triangular(1, 2, 3)	minutos	Mínimo 1, moda 2, máximo 3; media 2 min
Tiempo de prueba	Pruebas_Calidad (Service)	normal(2, 0.5)	minutos	σ = 2 y μ = 0,5 (ver sección de verificación)
Tipo de placa	llegadaPlacas, acción On exit	uniform(0, 1) < 0.25 ? "Alta" : "Baja"	-	25 % de placas de alta complejidad (supuesto)
Falla de mesa	Evento falloMesaCada8h	Primera ocurrencia en 240; recurrencia cada 480	minutos	Una falla dentro del turno simulado
Fin de reparación	Evento finReparacionMesa	Primera ocurrencia en 255; recurrencia cada 480	minutos	Reparación de 15 minutos
Horizonte	Experimentos	Stop at specified time: 480	minutos	Un turno de 8 horas

Tabla 3

Parámetros de configuración de escenarios (agente Main)

Parámetro	Tipo	Valor por defecto	Uso
escenarioOptimizado	boolean	false	Define la capacidad de los recursos:

Parámetro	Tipo	Valor por defecto	Uso
			Operarios_Ensamblaje = escenarioOptimizado ? 2 : 3; Mesa_Pruebas = escenarioOptimizado ? 2 : 1
habilitarFallas	boolean	true	Activa o desactiva la lógica de los eventos de falla y reparación

Variables del modelo

Tabla 4

Variables de salida y de estado definidas en Main

Variable	Tipo	Valor inicial	Descripción
totalLlegadas	int	0	Placas generadas por el Source
terminadas	int	0	Placas que llegaron al Sink
placasAlta	int	0	Placas de alta complejidad generadas
tiempoTotalSistema	double	0	Suma de los tiempos en el sistema de las placas terminadas
tiempoPromedioSistema	double	0	tiempoTotalSistema / terminadas
fallosMesa	int	0	Número de fallas de mesa ocurridas

Variable	Tipo	Valor inicial	Descripción
mesaEnReparacion	boolean	false	Estado actual de la mesa (true durante la reparación)

Además, dos series de datos registran Cola_Ensamblaje.size() y Cola_Pruebas.size() durante los 480 minutos y alimentan el gráfico de tiempo del modelo. El indicador de cola media usa la estadística interna Cola_Pruebas.statsSize.mean().

Definición de agentes

El modelo tiene dos tipos de agente:

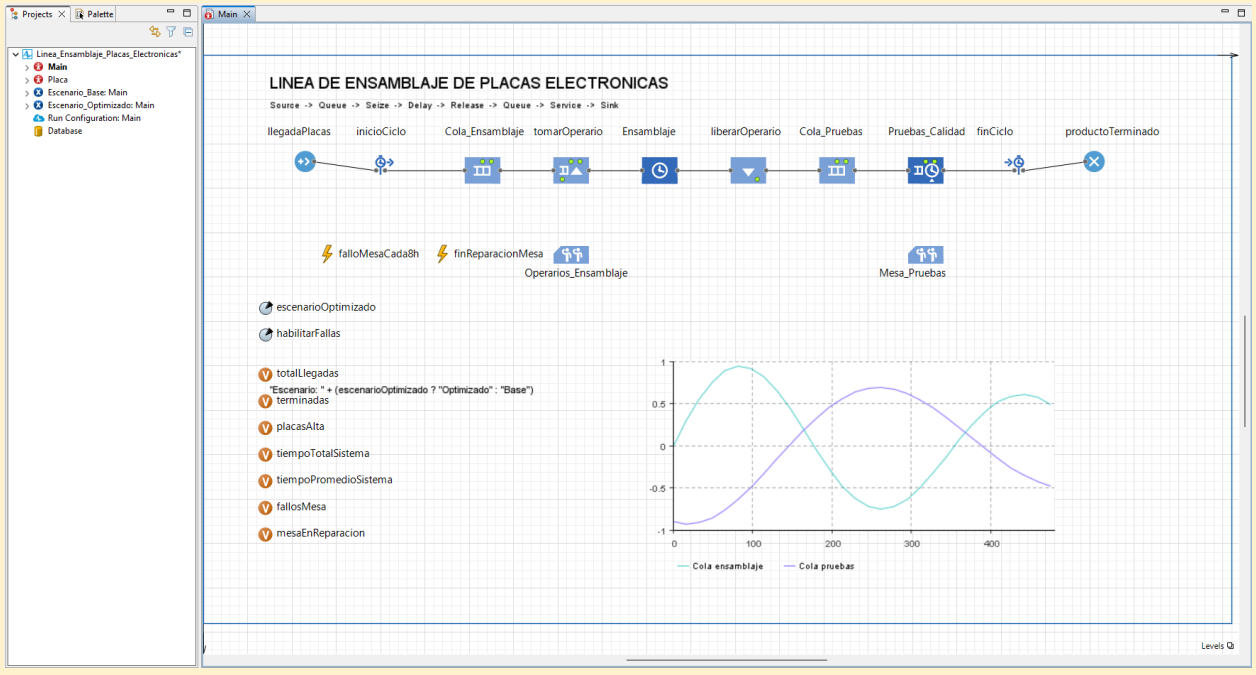
- **Main:** agente de nivel superior que contiene el diagrama de flujo, los dos ResourcePool, los eventos, los parámetros de escenario, las variables estadísticas, el texto de resultados y el gráfico. Su comportamiento consiste en coordinar el flujo de placas y actualizar los indicadores.
- **Placa:** agente que representa la entidad. Tiene las variables tipoPlaca (String, valor inicial "Baja") y horaLlegada (double, valor inicial 0). Su comportamiento es pasivo: recorre los bloques del diagrama de flujo, recibe sus atributos al salir del Source y aporta su tiempo de permanencia al llegar al Sink.

Diagrama de flujo del proceso

El flujo sigue la secuencia Source → TimeMeasureStart → Queue → Seize → Delay → Release → Queue → Service → TimeMeasureEnd → Sink. La Figura 1 muestra el diagrama construido y la Tabla 5 describe cada bloque.

Figura 1

Diagrama de bloques del modelo en la vista de edición de AnyLogic



Nota. Se observan el flujo de la entidad, las dos colas, los dos recursos, los eventos de falla y reparación, los parámetros de escenario y las variables estadísticas.

Tabla 5

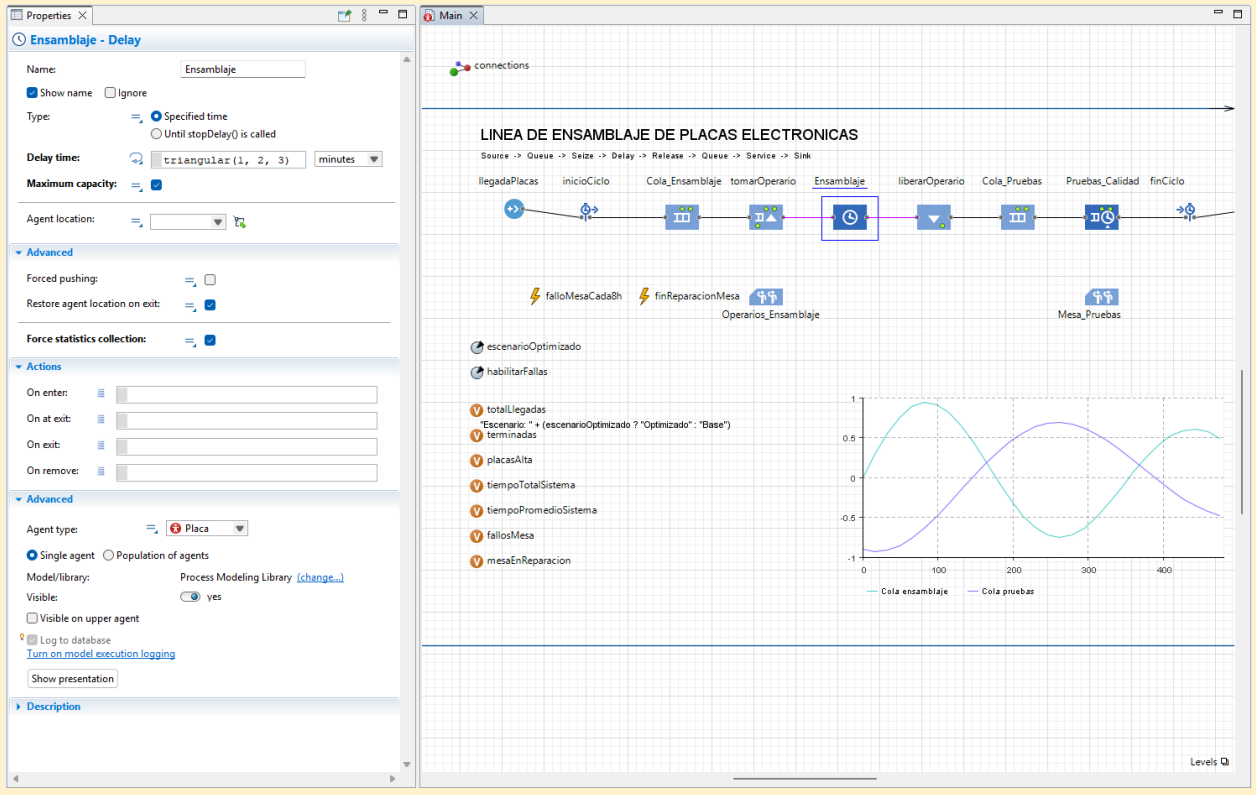
Bloques del diagrama de flujo y su configuración

Bloque	Tipo	Función y configuración
llegadaPlacas	Source	Genera agentes Placa con tiempo entre llegadas exponential(1.5). En On exit asigna horaLlegada y tipoPlaca y actualiza contadores.
inicioCiclo	TimeMeasureStart	Marca el inicio de la medición del tiempo de ciclo.
Cola_Ensamblaje	Queue	Capacidad máxima; almacena el WIP previo al ensamblaje.
tomarOperario	Seize	Toma una unidad del pool

Bloque	Tipo	Función y configuración
		Operarios_Ensamblaje.
Ensamblaje	Delay	Tiempo triangular(1, 2, 3) min; capacidad máxima, limitada en la práctica por los operarios disponibles (Figura 2).
liberarOperario	Release	Libera todos los recursos tomados (Release.ALL).
Cola_Pruebas	Queue	Capacidad máxima; almacena el WIP previo a pruebas.
Pruebas_Calidad	Service	Usa el pool Mesa_Pruebas; tiempo normal(2, 0.5) min.
finCiclo	TimeMeasureEnd	Cierra la medición iniciada en inicioCiclo.
productoTerminado	Sink	En On enter actualiza terminadas, tiempoTotalSistema y tiempoPromedioSistema.
Operarios_Ensamblaje	ResourcePool	Capacidad escenarioOptimizado ? 2 : 3.
Mesa_Pruebas	ResourcePool	Capacidad escenarioOptimizado ? 2 : 1; modificada por los eventos de falla.

Figura 2

Propiedades del bloque Delay Ensamblaje



Nota. El tiempo de retardo es triangular(1, 2, 3) en minutos, con capacidad máxima y recolección de estadísticas forzada.

Lógica de eventos

La lógica de eventos se programó en Java dentro de las acciones de los bloques y de los eventos. Al salir del Source, cada placa recibe sus atributos:

```
agent.horaLlegada = time();
agent.tipoPlaca = uniform(0, 1) < 0.25 ? "Alta" : "Baja";
totalLlegadas++;
if ("Alta".equals(agent.tipoPlaca)) placasAlta++;
```

Al entrar al Sink se actualizan los indicadores de salida:

```
terminadas++;
tiempoTotalSistema += time() - agent.horaLlegada;
tiempoPromedioSistema = tiempoTotalSistema / terminadas;
```

El evento falloMesaCada8h (cíclico, primera ocurrencia en el minuto 240) reduce en uno la capacidad de las mesas:

```
if (habilitarFallas) {
   fallosMesa++;
   mesaEnReparacion = true;
   Mesa_Pruebas.set_capacity((escenarioOptimizado ? 2 : 1) - 1);
}
```

El evento finReparacionMesa (cíclico, primera ocurrencia en el minuto 255) restablece la capacidad original:

```
if (habilitarFallas) {
   Mesa_Pruebas.set_capacity(escenarioOptimizado ? 2 : 1);
   mesaEnReparacion = false;
}
```

Estos dos eventos definen el diagrama de estados de la mesa de pruebas, resumido en la Tabla 6. En el escenario base la falla deja la estación de pruebas sin capacidad durante 15 minutos; en el optimizado queda una mesa operativa.

Tabla 6

Estados de la mesa de pruebas y transiciones

Estado	**Capacidad (base)**	**Capacidad (optimizado)**	**Transición de salida**
Operativa	1	2	falloMesaCada8h en t = 240 min → En reparación
En reparación	0	1	finReparacionMesa en t = 255 min → Operativa

Diseño de experimentos

Se definieron dos experimentos de simulación en AnyLogic que solo difieren en el valor

del parámetro escenarioOptimizado. La hipótesis era que la estación de pruebas, con una sola mesa, limitaba la producción, por lo que el escenario optimizado traslada un operario de ensamblaje (se mantiene el total de recursos humanos en la línea) y agrega una mesa de pruebas.

Tabla 7

Configuración de los experimentos

Configuración	Escenario_Base	Escenario_Optimizado
escenarioOptimizado	false	true
habilitarFallas	true	true
Operarios de ensamblaje	3	2
Mesas de pruebas	1	2
Duración	480 min	480 min
Generación de números aleatorios	Semilla fija = 1	Semilla fija = 1
Réplicas	1	1

Los indicadores de desempeño son: placas llegadas y terminadas, WIP al final del turno, tamaño de cada cola, cola media ante pruebas, tiempo medio en el sistema, producción por hora, fallas registradas y utilización de recursos.

Resultados

La Tabla 8 resume los indicadores al final de cada corrida y las Figuras 3 y 4 muestran el estado de la animación y la evolución de las colas.

Tabla 8

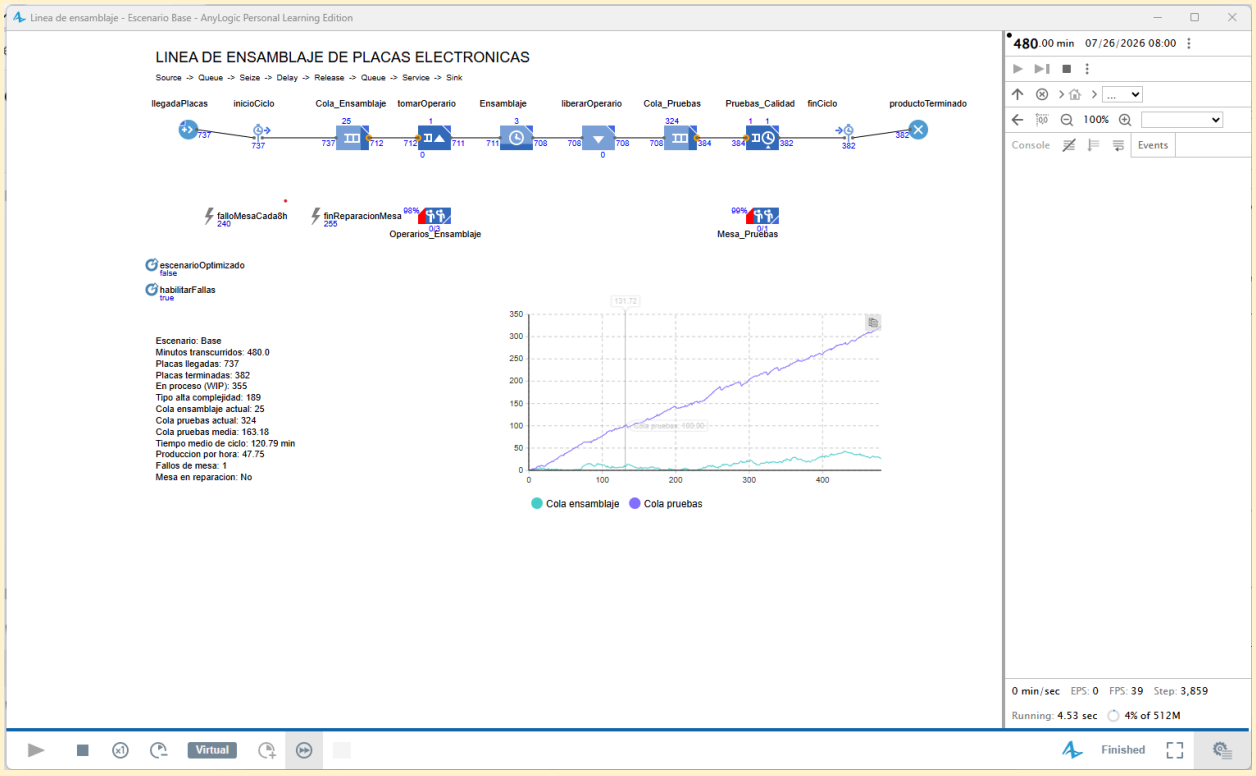
Resultados de los experimentos a los 480 minutos

Indicador	Base	Optimizado	Variación
Placas llegadas	737	724	-13
Placas terminadas	382	472	+90 (+23,6 %)
Placas en proceso (WIP)	355	252	-103 (-29,0 %)
Placas de alta complejidad	189 (25,6 %)	182 (25,1 %)	-
Cola de ensamblaje al final	25	246	+221
Cola de pruebas al final	324	0	-324
Cola media ante pruebas	163,18	0,08	-163,10
Tiempo medio en el sistema	120,79 min	71,53 min	-49,26 min (-40,8 %)
Producción por hora	47,75	59,00	+11,25
Utilización de operarios	98 %	99 %	+1 punto
Utilización de mesas de pruebas	99 %	54 %	-45 puntos
Fallas de mesa	1	1	-

Nota. Las utilizaciones son las que muestra AnyLogic en los ResourcePool al final de cada corrida (Figuras 3 y 4).

Figura 3

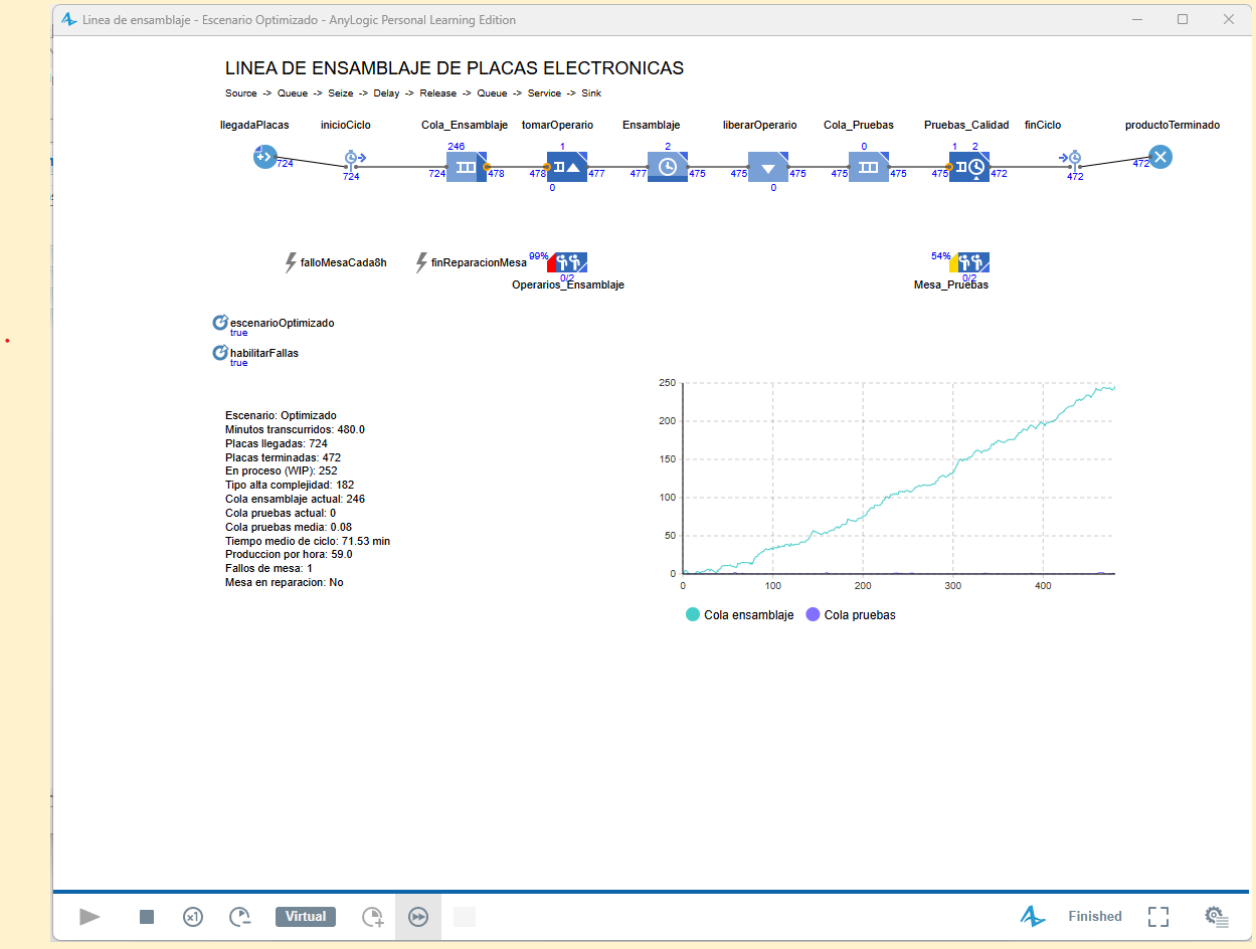
Ejecución del escenario base al finalizar los 480 minutos



Nota. La curva violeta (cola de pruebas) crece de forma casi lineal hasta 324 placas; la cola de ensamblaje (turquesa) se mantiene por debajo de 50.

Figura 4

Ejecución del escenario optimizado al finalizar los 480 minutos



Nota. La cola de pruebas permanece cerca de cero y la acumulación pasa a la cola de ensamblaje (turquesa), que llega a 246 placas.

Análisis y discusión

Comparación de escenarios

El escenario optimizado terminó 90 placas más en el turno (11,25 placas por hora más) y redujo el tiempo medio en el sistema en 49,26 minutos. La segunda mesa eliminó la acumulación ante pruebas: la cola media pasó de 163,18 a 0,08 placas y la falla del minuto 240 dejó de detener la estación, porque siempre quedó una mesa operativa.

Sin embargo, la mejora trasladó el cuello de botella al ensamblaje. Con dos operarios, la cola de ensamblaje llegó a 246 placas al final del turno y la utilización de los

operarios subió a 99 %, mientras que las mesas de pruebas quedaron ocupadas solo el 54 % del tiempo. Es decir, la segunda mesa pasa casi la mitad del turno sin trabajo porque el ensamblaje no le entrega placas suficientes.

Explicación del cuello de botella con teoría de colas

Con el tiempo medio de ensamblaje de 2 minutos, cada operario atiende 0,5 placas por minuto. En el escenario base, tres operarios tienen una capacidad de 1,5 placas/min, igual a la tasa de llegada; la utilización teórica es $\rho = 1{,}5 / 1{,}5 = 1$, por lo que la estación trabaja al límite y la cola fluctúa (terminó en 25 placas con una utilización observada de 98 %). En el escenario optimizado, dos operarios atienden 1,0 placa/min frente a 1,5 llegadas/min ($\rho = 1{,}5$). La cola debería crecer unas 0,5 placas por minuto, es decir, unas 240 placas en 480 minutos. El modelo registró 246, lo que coincide con el cálculo y confirma el comportamiento dinámico descrito en el marco teórico: la cola se acumula a la velocidad de la diferencia entre el flujo de entrada y el de salida.

La producción del escenario optimizado (59,0 placas/hora) también está muy cerca de la capacidad teórica del ensamblaje con dos operarios (60 placas/hora), lo que confirma que esa estación es la que limita el sistema.

Implicaciones

Ninguno de los dos escenarios es sostenible: en ambos el WIP crece durante todo el turno (355 y 252 placas al cierre). La mejora del escenario optimizado se debe a que la nueva restricción (60 placas/hora) es mayor que la restricción anterior en pruebas, no a que se haya equilibrado la línea. Como las mesas del escenario optimizado tienen capacidad ociosa, la alternativa lógica a evaluar es mantener tres operarios y agregar la segunda mesa. Ese escenario no se ejecutó en este trabajo y queda propuesto como siguiente experimento; antes de recomendarlo habría que incorporar el costo de cada recurso.

Verificación y limitaciones del modelo

Verificación

- **Balance de entidades:** en ambas corridas se cumple que llegadas = terminadas + WIP (737 = 382 + 355 y 724 = 472 + 252).
- **Mezcla de productos:** la proporción de placas de alta complejidad (25,6 % y 25,1 %) coincide con el 25 % configurado.
- **Capacidad:** la cola de ensamblaje del escenario optimizado (246) y su producción por hora (59,0) coinciden con los cálculos teóricos de la sección anterior.
- **Eventos:** se registró exactamente una falla por corrida, como corresponde a un horizonte de 480 minutos con primera falla en el minuto 240.

Observaciones sobre los parámetros

La revisión de la sintaxis de AnyLogic identificó dos puntos que conviene confirmar contra los datos del caso:

- **Llegadas:** exponential(1.5) usa 1,5 como tasa, es decir, 1,5 placas por minuto (0,67 min entre llegadas), lo que explica las más de 700 llegadas por turno. Si el caso define un tiempo medio entre llegadas de 1,5 minutos, la expresión correcta es exponential(1.0/1.5), con unas 320 llegadas por turno.
- **Pruebas:** normal(2, 0.5) usa desviación estándar 2 y media 0,5 minutos. Con esos valores, cerca del 40 % de las muestras serían negativas, lo que no representa un tiempo físico. Si se buscaba media 2 y desviación 0,5, la expresión es normal(0.5, 2), preferiblemente truncada, por ejemplo con max(0.1, normal(0.5, 2)).

Corregir cualquiera de estos parámetros cambiaría los resultados, por lo que ambos escenarios deberían volver a ejecutarse después del ajuste.

Limitaciones

- Cada escenario se ejecutó una sola vez. Las cifras describen esas corridas, pero no permiten estimar intervalos de confianza. Se recomienda ejecutar al menos 10

réplicas con semillas distintas.

- Las llegadas difieren entre escenarios (737 y 724) aunque ambos usan semilla fija 1. Los dos experimentos comparten un solo generador de números aleatorios; como cada escenario solicita muestras de tiempos de servicio en momentos distintos, la secuencia asignada a las llegadas también cambia. Usar flujos aleatorios independientes por proceso (números aleatorios comunes) haría la comparación más precisa.
- El tiempo medio en el sistema solo incluye placas terminadas. Como las colas crecen durante todo el turno, las placas que siguen en proceso tendrían tiempos mayores, por lo que el indicador subestima la permanencia real. Por la misma razón, el sistema no está en estado estable y la ley de Little no puede aplicarse directamente.
- La cola media ante pruebas corresponde al bloque Cola_Pruebas y no incluye la espera dentro del bloque Service.
- El atributo tipoPlaca no modifica los tiempos de proceso, no se modelan costos ni reprocesos, y el modelo no se validó con datos históricos de una línea real.

Conclusiones

- Se desarrolló en AnyLogic un modelo de eventos discretos que integra los cuatro elementos solicitados: colas (inventario en proceso), recursos, eventos programados y una entidad con atributos. Cada elemento se relaciona con los conceptos de entidad, atributo, evento y sistema dinámico de las Unidades I y II.
- En el escenario base, la mesa de pruebas única era el cuello de botella: su cola creció hasta 324 placas y la falla del minuto 240 detuvo por completo la estación durante 15 minutos.
- El escenario optimizado aumentó la producción en 23,6 % y redujo el tiempo medio en el sistema en 40,8 %, pero trasladó el cuello de botella al ensamblaje, donde dos operarios no alcanzan la tasa de llegada.
- Los resultados del modelo coinciden con los cálculos de capacidad de teoría de

colas, lo que respalda la lógica implementada. Antes de usar el modelo para decisiones, se deben confirmar los parámetros de llegadas y de pruebas, ejecutar réplicas y evaluar un tercer escenario con tres operarios y dos mesas.

Repositorio GIT

El proyecto completo se encuentra en:
https://github.com/tonatiuhcadenillas1-hub/assignment_2_simulation

Referencias

AnyLogic. (2024). *AnyLogic "How-To" Videos* [Video]. YouTube. https://www.youtube.com/playlist?list=PLUJJN9tmVTj1czFMt9IKi6wge9GnmMTN3

Banks, J., Carson, J. S., Nelson, B. L., & Nicol, D. M. (2010). *Discrete-event system simulation* (5.ª ed.). Pearson.

Barceló, J. (1996). *Simulación de sistemas discretos* (pp. 73-153). Isdefe.

Law, A. M. (2015). *Simulation modeling and analysis* (5.ª ed.). McGraw-Hill.

The AnyLogic Company. (s. f.). *AnyLogic: Software de simulación*. https://www.anylogic.com/

The AnyLogic Company. (s. f.). *exponential*. AnyLogic Help. https://anylogic.help/advanced/functions/exponential.html

The AnyLogic Company. (s. f.). *normal*. AnyLogic Help. https://anylogic.help/advanced/functions/normal.html